

Documento de Especificación del Proyecto: Sistema de Autómatas Finitos Deterministas

1. Visión General del Proyecto

El objetivo de este proyecto es desarrollar una herramienta de software orientada a la teoría de la computación que automatice dos tareas fundamentales: el análisis de validez formal de un **Autómata Finito Determinista (DFA)** y la ejecución de la **operación de unión** entre dos autómatas válidos.

El sistema garantiza rigor computacional mediante la verificación estricta de propiedades algebraicas de los DFA y la construcción precisa de un nuevo autómata derivado que acepte el lenguaje unión.

2. Restricción Técnica General: Implementación Manual de Estructuras de Datos

Para promover una comprensión profunda de las estructuras subyacentes, **queda estrictamente prohibido utilizar colecciones, estructuras de datos o librerías nativas/incorporadas del lenguaje de programación elegido** (como List, ArrayList, Vector, Set, HashSet, Map, Dictionary, HashMap, etc.), así como métodos o funciones auxiliares nativas para la manipulación compleja de datos (como .filter(), .map(), .reduce(), .indexOf(), .contains(), ordenamientos nativos o expresiones regulares complejas).

Requerimientos Obligatorios de Implementación:

- **Estructuras desde Cero:** Los estudiantes deberán diseñar e implementar manualmente desde cero las estructuras de datos necesarias para representar el autómata.
- **Algoritmos Propios:** Cualquier operación de búsqueda, inserción, eliminación, iteración o verificación de existencia dentro de los conjuntos (estados, alfabeto, estados finales y transiciones) debe ser programada explícitamente mediante algoritmos propios.

3. Requisitos Funcionales Detallados

Módulo 1: Validación Estricta de DFA

El sistema debe analizar un autómata ingresado por el usuario y evaluar de forma automatizada las siguientes verificaciones de validez sobre sus estructuras manuales:

1. Verificación de Alfabeto y Conjunto de Estados:

- **Unicidad:** Asegurar, mediante un algoritmo de búsqueda propio, que los nombres de los estados y los símbolos del alfabeto no contengan duplicados.
- **Simbología válida:** Impedir el uso de símbolos reservados o nulos (como ϵ , λ , espacios en blanco o guiones) dentro del alfabeto.

2. Verificación de Estado Inicial y Estados de Aceptación:

- **Existencia del inicial:** Validar mediante recorrido explícito que exista exactamente un estado inicial y que pertenezca al conjunto de estados.
- **Validez de estados finales:** Comprobar uno a uno que todos los estados marcados como de aceptación pertenezcan al conjunto de estados. Se permite que la lista/conjunto de estados de aceptación esté vacía.

3. Verificación de Determinismo y Totalidad en la Función de Transición:

- **Complejidad (δ total):** Iterar manualmente sobre **cada estado y cada símbolo del alfabeto** para comprobar que exista una y solo una transición definida para esa combinación.
- **Integridad de destino:** Garantizar que todo estado alcanzado por una transición sea un elemento registrado en la estructura de estados.
- **Prohibición de no-determinismo:** Rechazar el autómata si un par (estado, símbolo) apunta a múltiples estados o carece de destino en la estructura de transiciones.

4. Diagnóstico y Reporte de Errores:

- Si la validación falla, el sistema no permitirá guardar el autómata ni usarlo en operaciones posteriores.
- Debe generarse un informe de errores detallado señalando la falla exacta (por ejemplo: *"El estado 'q2' carece de transición para el símbolo 'b'" o "El estado de destino 'q5' no está registrado en el conjunto de estados"*).

Módulo 2: Operación de Unión entre Autómatas

Este módulo permite seleccionar dos DFA previamente validados y generar automáticamente la estructura del DFA resultante de su unión.

1. Criterios de Compatibilidad de Entrada:

- **Coincidencia de Alfabeto:** El sistema recorrerá manualmente los alfabetos de ambos autómatas para verificar que contengan exactamente los mismos símbolos.
- **Gestión de discrepancias:** Si los alfabetos difieren, el sistema debe bloquear la operación e indicar de manera específica qué símbolos no coinciden.

2. Generación del Autómata Resultante:

- **Mapeo de Estados Combos:** Cada nuevo estado estará representado explícitamente en la estructura mediante la combinación de un par (estado_DFA_1, estado_DFA_2).
- **Cálculo de Transiciones Combinadas:** Para cada nuevo estado par y cada símbolo del alfabeto compatible, la transición debe calcularse evaluando de forma simultánea (mediante búsqueda en las estructuras de transiciones originales) dónde se desplaza el primer autómata y dónde el segundo.
- **Criterio de Aceptación por Disyunción (OR):** Un estado compuesto (qA, qB) se registrará en la estructura de estados finales del nuevo autómata si qA es final en el primer autómata o qB es final en el segundo autómata (evaluado con algoritmos de búsqueda propios).

4. Salidas Esperadas y Visualización

Para proporcionar una experiencia completa e informativa, el sistema generará las siguientes salidas partiendo exclusivamente del recorrido de sus estructuras de datos propias:

1. Visualización y Mapeo del Autómata Resultante:

- **Matriz/Tabla de Transiciones:** Generación de una tabla que explicita los pares de estados resultantes, las transiciones para cada símbolo y los marcadores visuales del estado inicial (q0_A, q0_B) y de los estados de aceptación.
- **Lista Explícita de Componentes:** Resumen imprimible obtenido al iterar la estructura manual, mostrando el nuevo conjunto total de estados pares, el alfabeto utilizado, el estado inicial y la lista de todos los estados de aceptación identificados.

2. Módulo de Prueba e Inspección de Cadenas:

- **Ingreso de Cadenas de Evaluación:** Permitir al usuario digitar cualquier secuencia de símbolos perteneciente al alfabeto para probar su procesamiento.
- **Trazabilidad de Pasos:** Recorrer paso a paso la cadena de entrada evaluando las transiciones en la estructura del autómata unión, imprimiendo la secuencia de estados compuestos desde el estado inicial hasta el estado final alcanzado.
- **Veredicto de Aceptación Triple:** Indicar en pantalla el resultado final de evaluación de la cadena para tres casos simultáneos:
 - ¿Fue aceptada por el Autómata 1? (Aceptada / Rechazada)
 - ¿Fue aceptada por el Autómata 2? (Aceptada / Rechazada)
 - ¿Fue aceptada por el Autómata Unión Resultante? (Aceptada / Rechazada)

Rúbrica de Evaluación del Proyecto: Sistema de Autómatas Finitos Deterministas

Tabla Criterios de Evaluación

Criterio	Excelente (100%)	Satisfactorio (75%)	Suficiente (50%)	Insuficiente (0%)	Puntuación
1. Implementación Manual de Estructuras de Datos (25%)	Diseña e implementa 100% desde cero las estructuras (listas enlazadas, matrices, tablas Hash, etc.) y algoritmos de búsqueda/inserción. No utiliza ninguna colección, librería ni método nativo del lenguaje (List, Map, .contains(), etc.).	Implementa estructuras propias, pero utiliza de forma aislada 1 o 2 funciones/métodos nativos para búsquedas o iteraciones simples en el código.	Construye estructuras propias de forma parcial, apoyándose de forma recurrente en colecciones nativas o arreglos dinámicos del lenguaje.	Usa abiertamente colecciones nativas (List, ArrayList, Map, HashSet, etc.) o métodos nativos de manipulación compleja de datos.	<u> </u> /25
2. Módulo de Validación Estricta de DFA (25%)	Valida de forma impecable y automatizada los 4 criterios de un DFA (no vacuidad, unicidad, estado inicial/finales válidos, totalidad/completitud δ	Valida correctamente todos los criterios principales, pero la especificación del reporte de error es genérica o poco	Detecta errores básicos (como un estado inicial no existente), pero omite validaciones críticas como la totalidad de la	No valida los autómatas o permite guardar y procesar autómatas no deterministas/incompletos como si fueran DFA válidos.	<u> </u> /25

Criterio	Excelente (100%)	Satisfactorio (75%)	Suficiente (50%)	Insuficiente (0%)	Puntuación
	y no-determinismo). Genera reportes de error altamente específicos.	explícita ante fallas específicas.	función de transición δ o no-determinismo.		
3. Algoritmo y Operación de Unión (25%)	Genera de forma exacta el DFA unión mediante el producto cartesiano de estados (qA, qB), calcula transiciones compuestas y evalúa correctamente la disyunción (OR) de aceptación. Valida paridad de alfabetos.	Ejecuta la unión correctamente en la mayoría de casos, pero falla en casos borde (ej. autómatas con estados de aceptación vacíos o alfabetos dispares no bloqueados adecuadamente).	Construye los pares de estados, pero comete errores en la asignación de las transiciones compuestas o calcula mal el conjunto de estados finales (usa AND en lugar de OR).	No implementa la operación de unión o el algoritmo produce resultados incorrectos e incoherentes.	<u> </u> /25
4. Salidas, Trazabilidad y Prueba de Cadenas (15%)	Muestra con claridad la tabla/matriz del autómata unión. Permite probar cadenas mostrando el recorrido paso a paso y emite el	Muestra la matriz de transiciones y procesa cadenas con el veredicto triple, pero carece de la trazabilidad	Procesa la cadena en el autómata resultante pero no muestra la comparación	No permite evaluar cadenas o las evaluaciones de aceptación/rechazo arrojan resultados erróneos.	<u> </u> /15

Criterio	Excelente (100%)	Satisfactorio (75%)	Suficiente (50%)	Insuficiente (0%)	Puntuación
	veredicto triple (Autómata 1, Autómata 2 y Unión) de forma exacta.	paso a paso del recorrido de estados.	con los autómatas individuales, o la presentación visual es confusa.		
5. Calidad del Código y Presentación (10%)	Código limpio, modular, bien documentado y estructurado. Demuestra dominio total durante la defensa/demostración respondiendo con solvencia sobre el funcionamiento de sus estructuras manuales.	Código organizado y legible. Responde adecuadamente durante la defensa, aunque muestra dudas menores sobre el funcionamiento de algún algoritmo propio.	Código poco modular o desorganizado (difícil de seguir). Muestra dificultad para explicar cómo funcionan sus estructuras de datos implementadas.	Código desordenado, sin modularidad. No es capaz de explicar el funcionamiento del código ni de defender la autoría del trabajo.	___/10

Calificación Final: _____ / 100 pts.